第7章 第4节 Resources 实战

- 来源: https://class.imooc.com/lesson/5279#mid=104933
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: Resources概念与实战
- 课节: 图文 1-2 Resources 实战

1-2 Resources 实战

第7章 第4节 Resources 实战

> [!TIP] 阅读指南 上一节讲解了 Resources 的核心概念和工作原理。本节通过完整的实战示例，展示 Resources 的两种实现方式，以及如何编写高效的 description。

4.1 Resources 的实战示例

Resources 有两种生成方式：固定写死和动态生成，各有适用场景。

方式	适用场景	优点	缺点
固定写死	资源不常变化（如项目文档）	简单直接，易于理解	资源变化需修改代码
动态生成	资源动态变化，如数据库表、API	自动同步，无需手动维护	需要额外查询逻辑

下面通过两个示例来看具体实现。

API 文档 Server

为项目的 API 文档创建一个 MCP Server，让 AI 能够查询接口使用说明。

实现思路

```
# ═══════════════════════════════════
#  API文档 MCP Server（伪代码示意）
# ═══════════════════════════════════

class APIDocServer:
    def list_resources(self):
        # ▸ 返回所有可用的 Resources
        return [
            {
                "uri": "file://docs/authentication.md",
                "name": "用户认证API文档",
                "description": "用户认证相关API：登录、注册、登出、Token刷新",
                "mimeType": "text/markdown"
            },
            {
                "uri": "file://docs/user_profile.md",
                "name": "用户资料API文档",
                "description": "用户资料管理API：获取信息、修改头像、更新昵称",
                "mimeType": "text/markdown"
            },
            {
                "uri": "file://docs/payment.md",
                "name": "支付API文档",
                "description": "支付相关API：创建订单、查询状态、退款处理",
                "mimeType": "text/markdown"
            },
            # ... 更多资源
        ]

    def read_resource(self, uri):
        # ▸ 读取指定 Resource 的内容
        file_path = uri.replace("file://", "")
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()

        return {
            "contents": [{
                "uri": uri,
                "mimeType": "text/markdown",
                "text": content
            }]
        }
```

工作流程

```
用户："如何实现用户登录功能?"
  ↓
1. AI 应用调用 list_resources()
  获取：[用户认证API文档， 用户资料API文档， ...]
  ↓
2. 传给 LLM（附带所有 descriptions）
  ↓
3. LLM 判断："用户认证API文档" 最相关
  ↓
4. AI 应用把请求转给 MCP Server，由 Server 调用
read_resource("file://docs/authentication.md")
  ↓
5. 返回文档内容给 LLM
  ↓
6. LLM 阅读内容，回答用户
```

数据库 Schema Server

> 重点：Resources 可以动态生成！ 上一个示例（API文档 Server）中，Resources 是固定写死的： 文档文件名固定：authentication.md、user_profile.md Resources 列表不变，除非手动修改代码 但很多场景下，资源是动态变化的： 数据库表可能随时增加 API 端点可能按版本变化 用户文件可能动态上传 这时候，动态生成 Resources 列表是更好的选择！

让 AI 能够查询数据库的表结构、索引等元信息。

Resources 设计

```
# 注意：以下为伪代码示意，不可直接运行

class DatabaseSchemaServer:
    def list_resources(self):
        # ▸ 动态生成 Resources（关键！）
        resources = []
        tables = self.get_all_tables()
        for table in tables:
            resources.extend([
                {"uri": f"postgres://{table}/schema", "name": f"{table}表结构",
"description": "..."},
                {"uri": f"postgres://{table}/indexes", "name": f"{table}表索引",
"description": "..."},
                {"uri": f"postgres://{table}/sample", "name": f"{table}示例数据",
"description": "..."}
            ])
        return resources
        # 假设数据库有 users、orders 两个表
        # 就会动态生成 6 个 Resources：
        # - postgres://users/schema
        # - postgres://users/indexes
        # - postgres://users/sample
        # - postgres://orders/schema
        # - postgres://orders/indexes
        # - postgres://orders/sample

    def read_resource(self, uri):
        # ▸ 解析 URI,执行相应查询
        parts = uri.split('/')
        table = parts[1]
        resource_type = parts[2]

        if resource_type == "schema":
            # 查询表结构
            result = self.get_table_schema(table)
        elif resource_type == "indexes":
            # 查询索引信息
            result = self.get_table_indexes(table)
        # ...

        return {"contents": [{"text": result}]}
```

使用场景

```
用户: "users表有哪些索引?"
  ↓
LLM 选择: postgres://users/indexes
  ↓
Server 执行: SELECT * FROM pg_indexes WHERE tablename='users'
  ↓
返回索引信息
```

4.2 编写优秀的 description

description 是 Resources 的灵魂!写得好不好直接影响 LLM 能否找对资源。

好的 description 的特征

清晰描述内容

```
× 不好:
"description": "认证文档"

✓ 好:
"description": "API认证方式说明，包括JWT Token的获取、刷新和验证流程"
```

包含关键词

```
× 不好:
"description": "关于系统的一些配置"

✓ 好:
"description": "系统配置文件，包括数据库连接、Redis缓存、日志级别等环境变量设置"
```

区分度高

假设有两个文档：

× 区分度低：
- "用户管理文档"
- "用户相关功能"

✓ 区分度高：
- "用户管理：创建、修改、删除用户的API接口"
- "用户权限：角色分配、权限检查的实现逻辑"

常见 description 模板

文档类 Resource

格式：<主题> - <核心内容>,<关键细节>

示例：
"API认证 - JWT Token实现，包括token生成、验证和刷新流程"
"数据库设计 - 表结构定义，涵盖users、orders等核心表及其关系"
"部署指南 - Docker容器化部署，包括镜像构建、环境配置和启动命令"

数据类 Resource

格式：<数据类型>的<属性/操作>,<范围/条件>

示例：
"users表的列定义和约束条件"
"订单数据的前100条示例记录，按时间倒序"
"2024年1月的销售统计报表"

API 类 Resource

格式：<操作> <对象> - <功能说明>

示例：
"POST /users - 创建新用户，需要username、email、password参数"
"GET /users/{id} - 获取指定用户的详细信息"
"PUT /users/{id}/password - 修改用户密码，需要旧密码验证"

4.3 Resources 也有够不着的地方

资源数量太多

问题：
如果有 10000 个文档，生成 10000 个 Resource description
→ description 列表本身就太长，LLM 看不完

解决方案：
- 分层设计：先选择分类，再检索具体文档
- 或者用 RAG

资源边界模糊

问题：
如果文档内容互相重叠，description 很难写清楚

示例：
- "用户管理。md" - 包含用户CRUD和权限
- "权限系统。md" - 也包含用户权限相关内容

→ 用户问"如何给用户分配权限?"
→ 两个文档都相关，LLM 难以选择

解决方案：
- 重构文档，让边界清晰
- 或者允许返回多个 Resources

需要模糊搜索

问题：
用户问："有没有关于性能优化的内容？"

Resources 需要：
- description 中包含"性能优化"关键词
- 否则 LLM 找不到

向量检索：
- 可以找到语义相关的内容
- 即使没有"性能优化"这个词

结论：模糊搜索还是 RAG 更强

4.4 下一节预告

到这里，Resources 的实战技巧已经讲清楚了：从固定写死的 API 文档 Server 到动态生成的数据库 Schema Server，再到精心编写 description 提升匹配精度。下一节将把视角从「查资料」扩展到「做事情」——系统讲解 MCP 中的 Tools，如何让 AI 不仅读取数据，还能安全地调用工具、执行操作。

4.5 ■ 学点英语

中文	English	音标	说明
固定写死	Hardcoded Resources	/ˈhɑːrdˈkoʊdɪd rɪˈsɔːrsɪz/	在代码中预定义Resources，适用于资源不常变化的场景
动态生成	Dynamic Generation	/daɪˈnæmɪk ˌdʒenəˈreɪʃn/	实时查询外部系统自动生成Resources列表的方式
Description 编写	Description Authoring	/dɪˈskrɪpʃn ˈɔːθərɪŋ/	编写Resource描述以帮助LLM精准匹配资源的技术
轮询刷新	Polling Refresh	/ˈpoʊlɪŋ riːˈfreʃ/	定时扫描外部系统以更新Resources列表的机制